

结构化编程思想

面向对象编程 · 2026 · 南京大学

| 刘钦

Outline

本讲是结构化编程三讲的第一讲（设计层面），后续两讲分别讲数据（变量）和行为（方法）。

- 什么是结构化编程 — Dijkstra · Bohm-Jacopini 定理
- 思想和模型 — 自顶向下逐步求精 · 算法+数据结构
- 数据流图 — 世界观 · 图例 · 语法规则 · 画图流程
- 结构图 — 图例 · DFD向结构图的转换
- 流程图 — 基本符号 · 案例

什么是结构化编程

Dijkstra · Bohm-Jacopini · 三种结构

结构化编程的起源

1968年，Dijkstra 发表 "**Go To Statement Considered Harmful**", 主张用结构化控制流替代 goto 语句。

同年，Bohm 和 Jacopini 证明了：

任何可计算的算法，都可以仅用顺序、分支、循环三种控制结构表达。

结构	含义	对应
顺序	从上到下逐行执行	语句依次排列
分支	根据条件选择路径	if / switch
循环	重复执行直到条件不满足	for / while

结构化编程 = 自顶向下分解 + 三种控制结构 + 模块化设计

结构化方法的历史沿革

年代	里程碑	意义
1966	Bohm–Jacopini 定理	证明三种结构（顺序/分支/循环）的计算完备性
1968	Dijkstra "Go To Considered Harmful"	掀起结构化编程运动，goto 开始被淘汰
1969	Wirth 提出逐步求精 (Stepwise Refinement)	自顶向下分解方法的理论基础
1971	Wirth 发表 "Program Development by Stepwise Refinement"	经典论文，奠定结构化设计方法论
1974	Stevens, Myers, Constantine 提出结构化设计	模块化 · 耦合 · 内聚概念形成
1975	Wirth 出版 Algorithms + Data Structures = Programs	结构化编程思想的集大成之作
1978	DeMarco 提出结构化分析	数据流图(DFD)成为标准建模工具
1979	Yourdon & Constantine Structured Design	结构图(SC)方法成熟

结构化方法在 1970s–1980s 是软件开发的主流范式，直到面向对象方法在 1990s 逐渐取代它。

结构化方法的后续发展

年代	发展	意义
1980s	结构化分析与设计技术 (SADT/IDEF0)	从软件扩展到 系统工程 ，用于业务流程建模
1985	Hatley–Pirbhai 实时扩展	在 DFD 基础上加入 控制流图 ，支持嵌入式/实时系统
1988	Ward–Mellor 实时结构化分析	结构化方法进入航空航天、国防领域
1989	SSADM (英国政府标准)	结构化方法成为 政府级 系统开发标准
1990s	面向对象方法兴起 (Booch, OMT, OOSE)	结构化方法开始被 OOP 取代，但 DFD 仍在需求分析中广泛使用
1997	UML 1.0 统一建模语言	OO 方法统一，结构化方法 逐渐边缘化
2000s	BPMN (业务流程建模)	DFD 的思想以 新形式 在业务流程管理中延续

结构化方法并没有"消亡"——它的核心思想（分解、数据流、模块化）被 OOP 和现代工程方法**吸收和演化**。

思想和模型

自顶向下逐步求精 · 算法+数据结构

结构化方法

- 思想

- 自顶向下逐步求精 (Wirth, 1969)
- 算法 + 数据结构 = 程序 (Wirth, 1975)

- 模型

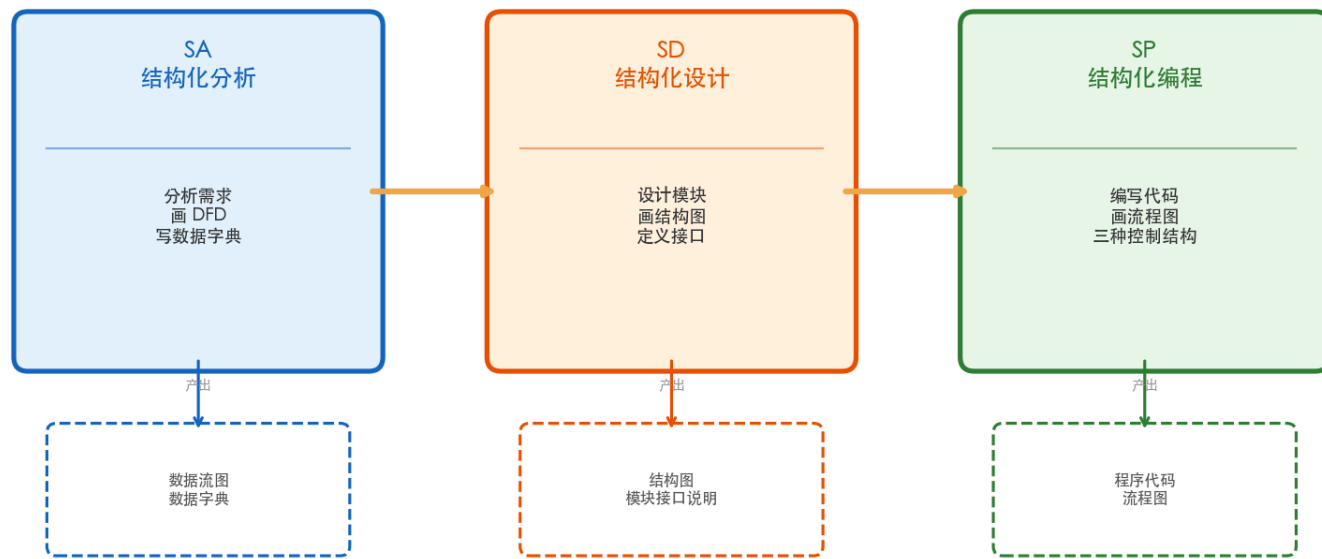
- 数据流图 (DeMarco, 1978) — 分析阶段
- 结构图 (Yourdon & Constantine, 1979) — 设计阶段
- 流程图 — 实现阶段

- 开发流程

- 结构化分析 (SA) → 结构化设计 (SD) → 结构化编程 (SP)

结构化开发流程

结构化开发流程：SA → SD → SP



SA → SD → SP 是瀑布模型在结构化方法中的具体体现。每个阶段有明确的输入、活动、产出。

数据流图

世界观 · 图例 · 语法规则 · 画图流程

数据流图的世界观

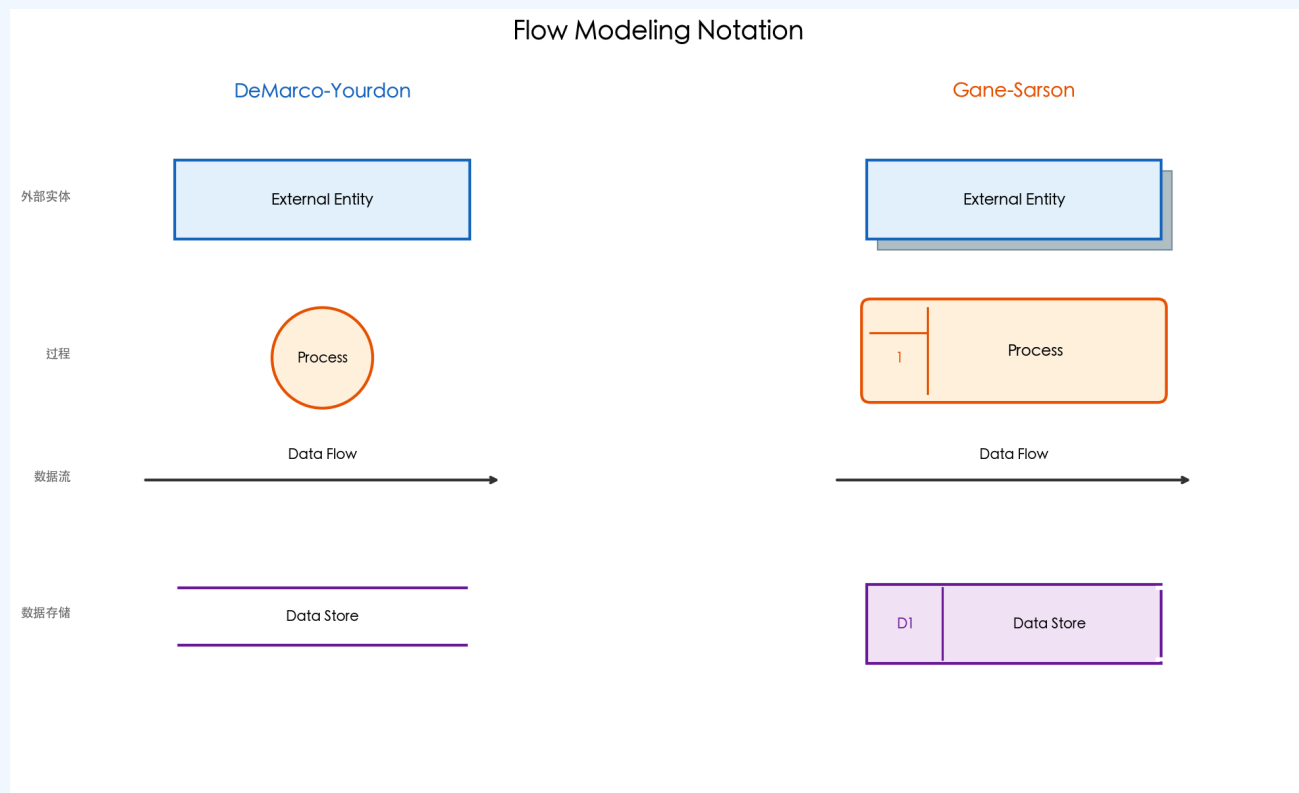
输入 → [计算系统] → 输出

所有的计算系统都是信息的处理和转换。

过程与数据

- 将系统看做是过程的集合；
- 过程就是对数据的处理：
 - 接收输入，进行数据转换，输出结果
 - 代表数据对象在穿过系统时如何被转换
- 可能需要和软件系统外的实体尤其是人进行交互
- 数据的变化包括：
 - 被转换、被存储、或者被分布

数据流图 — 图例 (1/2)



外部实体

- 数据的生产者或者消费者（人、设备、传感器、计算机系统）
- 数据总是从某处来，然后流向其它的地方

过程

数据流图 — 图例 (2/2)

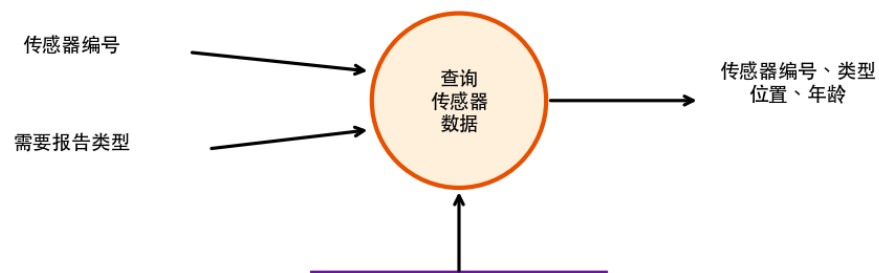
数据流

- 通过系统的数据流总是从输入被转换为输出



数据的存储

- 数据有时候会被存储起来为以后使用做准备。



数据流图 — 语法规则

- 过程是对数据的处理，必须有输入，也必须有输出，输入数据集应该和输出数据集存在差异
- 数据流是必须和过程产生关联的，它要么是过程的数据输入，要么是过程的数据输出
- 所有的对象都应该有一个可以唯一标示自己的名称

错误：过程无输入 / 过程无输出 / 输入输出相同 / 外部实体直接连外部实体 / 外部实体直接连数据存储

正确：所有数据流必须经过过程，过程有输入且有输出，且输入输出不同

理解数据流图

- 处理
 - 并不一定是程序。
 - 它可以是一系统程序、单个程序或程序的一个模块，甚至可以是人工处理过程；
- 数据存储
 - 并不等同于一个文件。
 - 它可以是一个文件、文件的一部分、数据库元素或记录的一部分；它代表的是静态的数据。
- 数据流
 - 也是数据，是动态的数据。

数据字典（Data Dictionary）

数据字典是对 DFD 中所有数据流、数据存储、数据项的精确定义，是 DFD 的配套文档。

DFD 说明"数据从哪来、到哪去"；数据字典说明"数据是什么"。

符号	含义	示例
=	被定义为	定货报表 = {定货记录}
+	与（连接）	定货记录 = 零件编号 + 零件名称 + 数量 + 价格
[... ...]	或（选择）	事务类型 = [入库 出库]
{...}	重复	定货报表 = {定货记录}
(...)	可选	供应商 = 名称 + 地址 + (电话)
...	基本数据项	零件编号 = **6位数字**

数据字典 — 案例

以订货系统为例：

事务 = 零件编号 + 事务类型 + 数量

事务类型 = [入库 | 出库]

零件编号 = **6位数字字符**

库存清单 = {库存记录}

库存记录 = 零件编号 + 库存量 + 库存量临界值

定货信息 = {定货记录}

定货记录 = 零件编号 + 零件名称 + 定货数量
+ 目前价格 + 主要供应商 + 次要供应商

供应商 = 名称 + 地址 + (电话)

数据字典使 DFD 中的每个数据流和数据存储都有**唯一、明确、无歧义**的定义。

案例一订货系统

- 假设一家工厂的采购部门每天需要一张定货报表。报表按零件编号排序，表中列出所有需要再次定货的零件。
- 对于每个需要再次定货的零件应该列出下述数据：零件编号、零件名称、定货数量、目前价格、主要供应商、次要供应商。
- 零件入库或出库称为事务，通过放在仓库中的CRT终端把事务报告给定货系统。当某种零件的库存数量少于库存临界值时就应该再次定货。

案例

- 源点/终点（外部实体）
 - 采购员
 - 仓库管理员
- 数据处理
 - 产生报表
 - 处理事务
- 数据流
 - 定货报表（零件编号、名称、数量、价格）
 - 主要供应商、次要供应商
 - 事务（零件编号、事务类型、数量）
- 数据存储
 - 定货信息（见定货报表）
 - 库存清单（零件编号、库存量、库存量临界值）

订货系统 — 数据流图逐步细化

Level 0 — 最概括

订货系统 — 最概括的系统模型



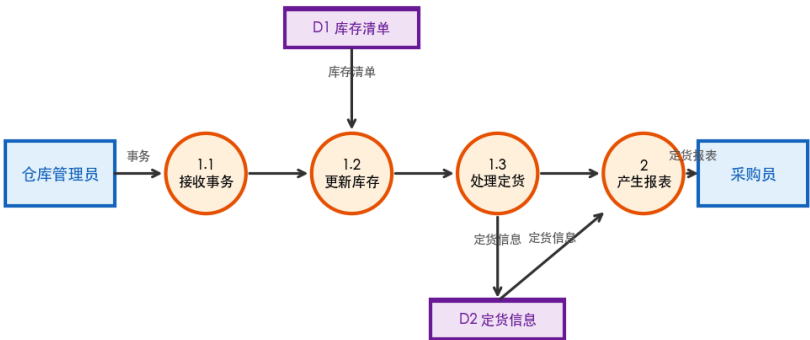
Level 1 — 细化数据存储和数据流

订货系统 — 细化数据存储和数据流



Level 2 — 子过程

订货系统 — 子过程



自顶向下逐步求精：从最抽象的系统模型出发，每一步细化一个层次，直到足够详细为止。

订货系统 — 边界与细化

子系统边界划分

- 子系统 1: 接收事务 → 更新库存清单 → 处理定货（包含 D1 库存清单、D3 事务）
- 子系统 2: 产生报表（使用 D2 定货信息）

子过程进一步细化

- 1.1 接收事务从仓库管理员获取事务数据，写入 D3 事务存储
- 1.2 更新库存清单读取 D1 库存清单进行更新
- 1.3 处理定货判断是否需要再次定货，写入 D2 定货信息

课程表案例

- 建一个课程表。
 - 星期四；三，四节；计算与软件工程；仙2-407；
- 通过命令行方式完成对课程的增、删、改、查、显示。
 - Add 星期四；三，四节；计算与软件工程；仙2-407； //如果成功 显示"已添加到文件中"
 - Remove 星期四；三，四节；计算与软件工程；仙2-407； //如果成功 显示"已从文件删除"
 - Update 星期四；三，四节；计算与软件工程；仙2-408； //如果成功 显示"已更新文件"
 - Find 星期四；三，四节； //如存在课程 显示 "课程名；上课地点"
 - Show //显示所有课程，按照时间排序
- 数据保存在文件里。
 - CurriculumSchedule.txt

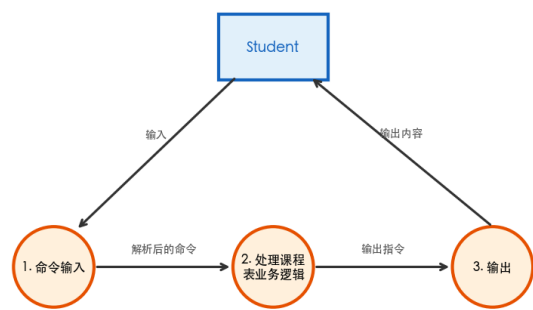
课程表案例分析

- 源点/终点（外部实体）
 - 学生
- 数据处理
 - 命令输入（输入、分析输入）
 - 处理课程表业务逻辑（增、删、改、查）
 - 显示 / 输出（生成输出内容、文件输出、控制台输出）
- 数据流
 - 命令
- 数据存储
 - 课程表数据（文件地址）

课程表 — 数据流图

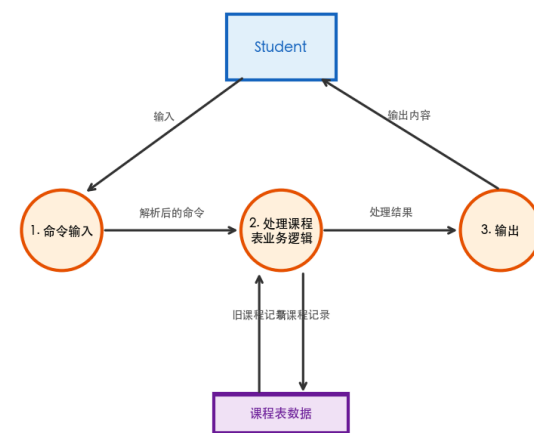
概括

课程表 — 数据流图（概括）



细化

课程表 — 数据流图（细化）



结构图

图例 · DFD向结构图的转换

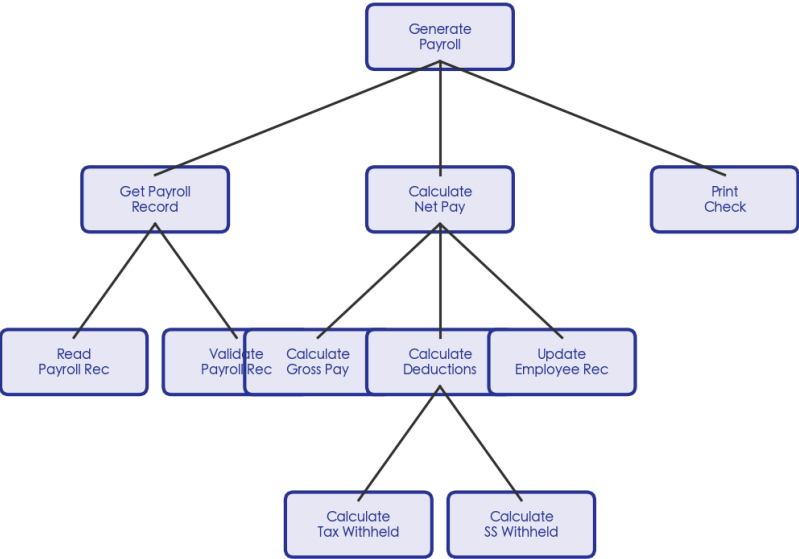
结构图（Structured Chart）

结构图展示模块间的调用关系和数据传递。

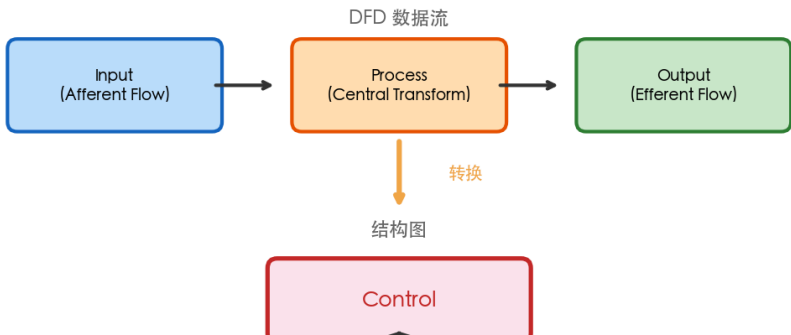
符号	含义
矩形	Modules（模块）
双竖线矩形	Library modules（库模块）
连线	Module call（模块调用）
空心圆+箭头	Data（数据传递）
实心圆+箭头	Flag（标志/控制信息传递）

转换分析与 Payroll 案例

结构图 (Structured Chart) — Payroll



转换分析：DFD → 结构图



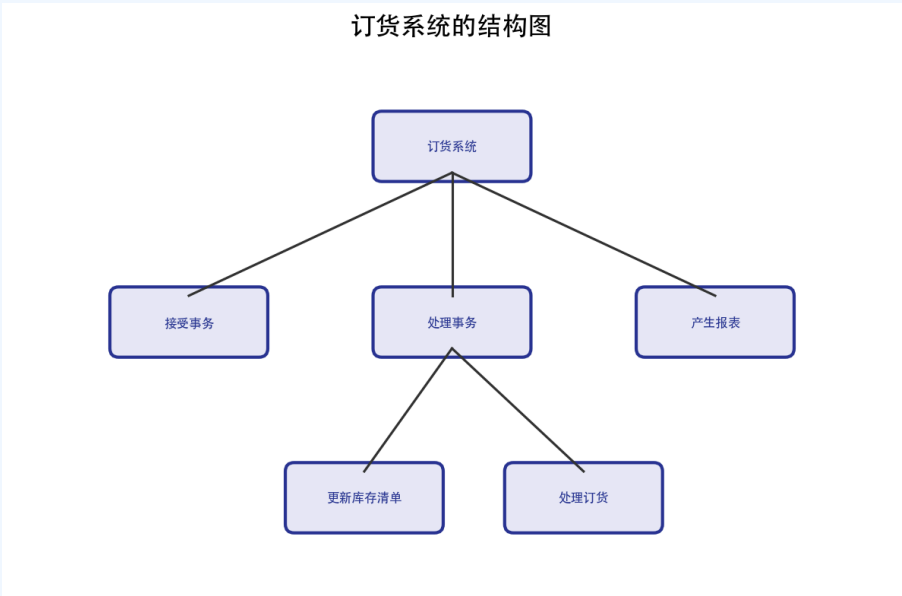
数据流图向结构图的转换

1. 在 DFD 中找到**输入最高抽象点**和**输出最高抽象点**
2. 将 DFD 划分为输入模块、转换模块、输出模块
3. 映射为结构图的三层结构

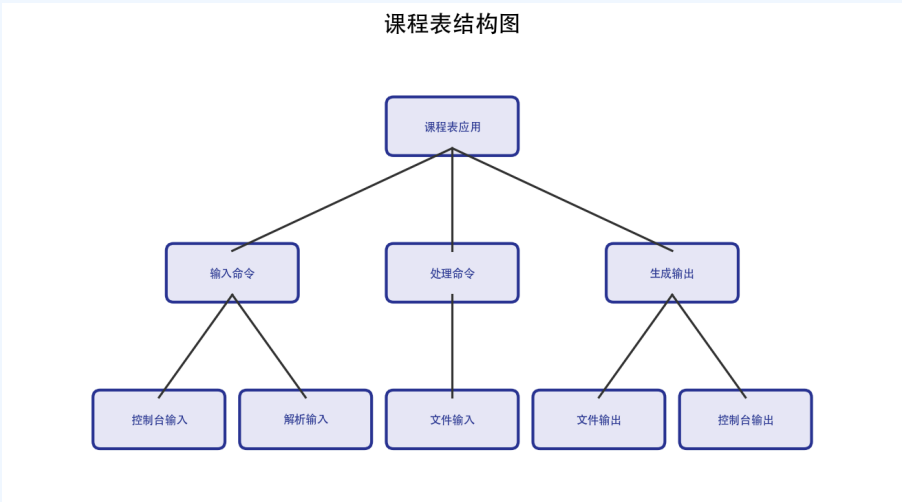
示例：登录模块 → 读入并验证（输入） → 数据库验证（转换） → 页面跳转（输出）

订货系统与课程表 — 结构图

订货系统



课程表



数据设计：文件名数据 · 命令数据 · 课程信息数据

结构图转换为代码

以 word count 案例为例：

```
def read_and_validate():  
    return validated_file_name, status_flag  
  
def count(validated_file_name):  
    return wordcount  
  
def format_display(wordcount):  
    pass  
  
def perform_word_count():  
    validated_file_name, status_flag = read_and_validate()  
    count = count(validated_file_name)  
    format_display(count)
```

流程图

基本符号 · 案例

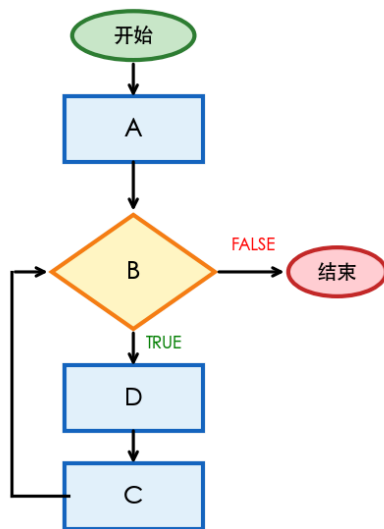
算法与流程图表示

算法

- 顺序结构 (Sequence): 一条一条执行
- 选择结构 (Selection): if / switch
- 循环结构 (Iteration): while / for

流程图 — for 循环

for(A; B; C) D;



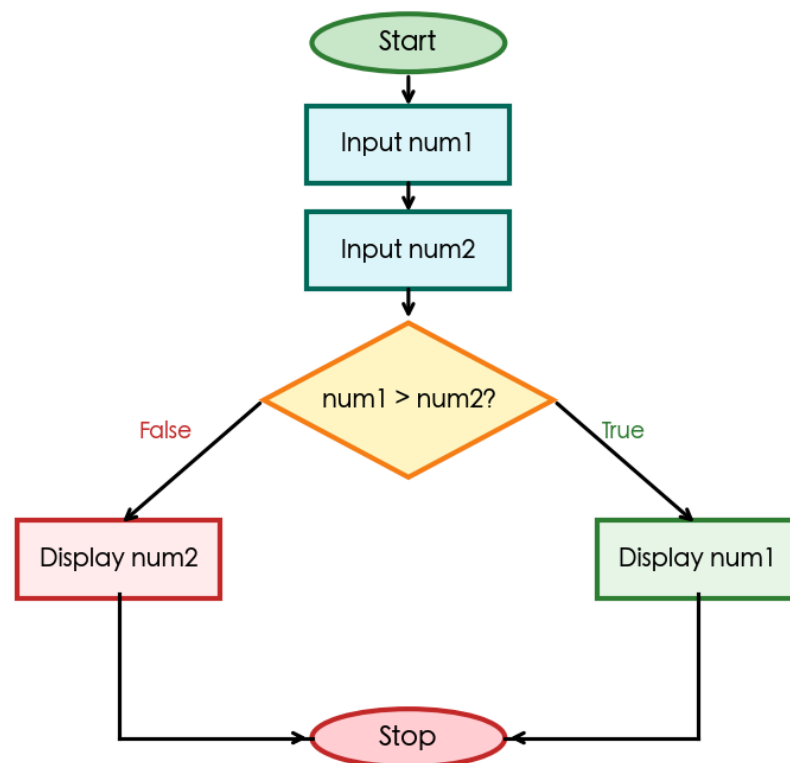
Flowchart — Basic Symbols

1. **Terminal:** The oval symbol indicates Start, Stop and Halt in a program's logic flow.
2. **Input/Output:** A parallelogram denotes any function of input/output type.
3. **Processing:** A box represents arithmetic instructions.
4. **Decision:** Diamond symbol represents a decision point (yes/no or true/false).
5. **Connectors:** Circle, used when flowchart spreads over more than one page.
6. **Flow lines:** Arrows represent the direction of flow of control.

Example — 找出两个数中的较大值

```
import java.util.Scanner;
public class Largest {
    public static void main(String args[]) {
        int num1, num2, max;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter two numbers:");
        num1 = sc.nextInt();
        num2 = sc.nextInt();
        if (num1 > num2)
            max = num1;
        else
            max = num2;
        System.out.println(max);
    }
}
```

找出两个数中的较大值



数据结构

- 课程表数据
 - 数组 or ArrayList
 - 全局变量
- 文件地址
 - char[] or String
 - 文件名作为常量

总结

结构化编程思想

- 思想：自顶向下逐步求精 · 算法+数据结构
- 开发流程：SA(结构化分析) → SD(结构化设计) → SP(结构化编程)
- 数据流图(DFD)
 - | 外部实体 · 过程 · 数据流 · 数据存储 · 数据字典
 - | 案例：订货系统(3级细化) · 课程表
- 结构图(SC)
 - | DFD → 转换分析(Input/Process/Output) → 结构图
 - | 案例：Payroll · 订货系统 · 课程表
- 流程图
 - | 六种基本符号 · for循环 · 找最大值

- **行为视角**：首先根据行为来分解，接着设计数据来配合行为（全局数据）
- **下一讲**：变量与数据类型——结构化编程的数据层面